

Camera-Invariant Domain Adaptation

EURON 8th project

참치팀 : 팽소원, 장효인, 이수나, 함예린

목차

01
프로젝트 주제

02
이론

03
사용한 모델

04
적용한 기법

05
실험 과정

06
결론

1. 프로젝트 주제

프로젝트 주제



- 왜곡이 없는 이미지와 대응되는 레이블 정보를 활용하여,
레이블이 존재하지 않는 왜곡된 영상에서도 semantic segmentation 인식을 수행하는 알고리즘 개발
- 즉, 왜곡이 없는 이미지 도메인을 사용해 왜곡이 있는 target.domain에서도
잘 작동하는 카메라 특성 변화에 강인한 domain adaptive semantic segmentation 알고리즘 개발을 목표

Semantic Segmentation

: 이미지 내에서 픽셀 단위로 객체(Object)를 분류

Object는 사람, 자동차, 나무, 자전거 등을 말하며, Deep learning에서 이를 class라고 표현

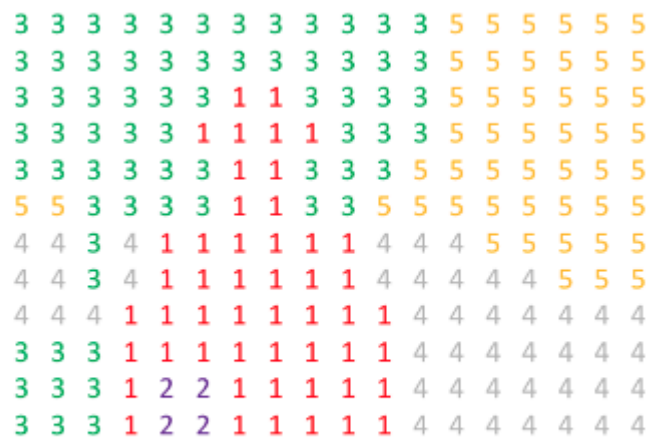
Class ID	Class Name	Pixel Value
0	Road	0
1	Sidewalk	1
2	Construction	2
3	Fence	3
4	Pole	4
5	Traffic Light	5
6	Traffic Sign	6
7	Nature	7
8	Sky	8
9	Person	9
10	Rider	10
11	Car	11



Input



- 1: Person
- 2: Purse
- 3: Plants/Grass
- 4: Sidewalk
- 5: Building/Structures



Semantic Labels



1. 프로젝트 주제

✓ 데이터셋

- train_source_image
- train_source_gt
- train_target_image
- val_source_image
- val_source_gt
- test_image

사용한 데이터셋



TRAIN_SOURCE_0011



TRAIN_SOURCE_0011
(GT IMAGE)



VAL_SOURCE_001



TEST_IMAGE



TARGET_IMAGE

2. 이론



TRAIN_SOURCE_0000



TEST_0000

Domain Adaption



도메인이 다르지만 관련이 있는 새로운 영역(target domain)에
기존 영역(source domain)의 정보를 적응시키는 것

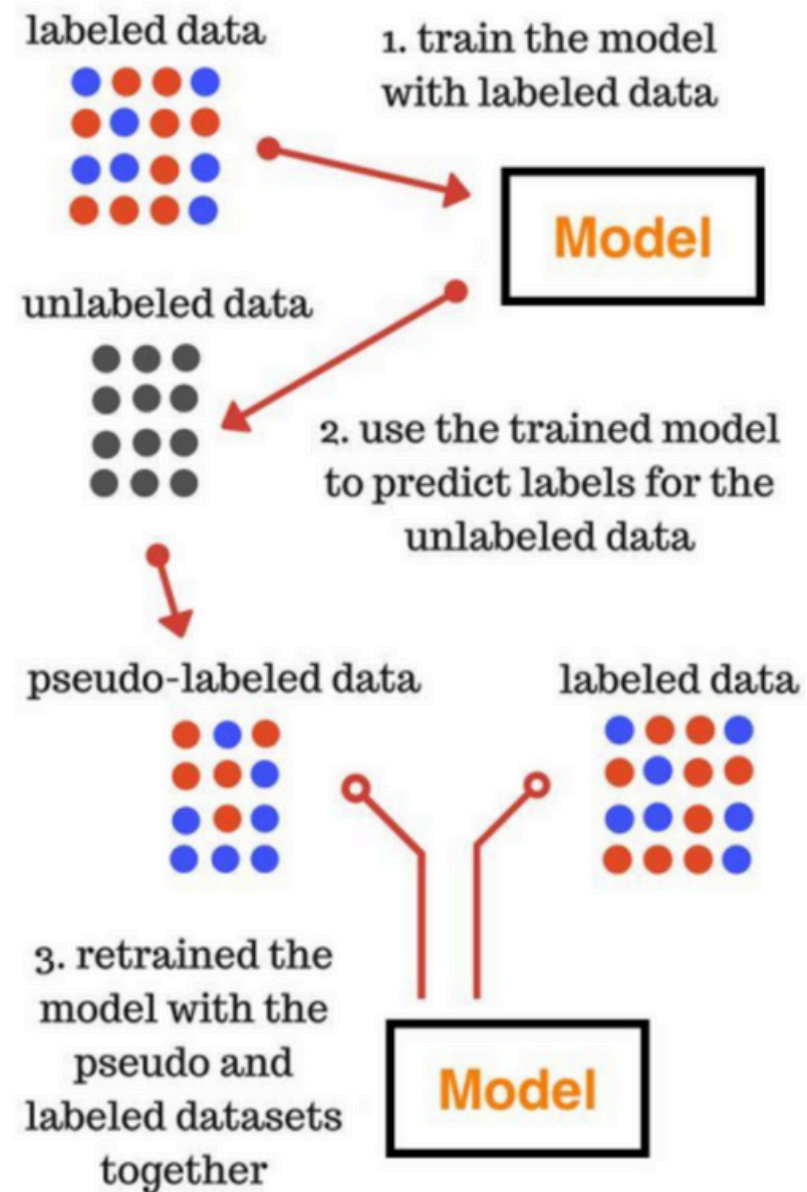
Domain shift 문제

- 학습 데이터와 테스트 데이터의 분포 차이를 의미
 - train data: 굴곡이 없는 일반 이미지
 - test data: 어안렌즈로 찍은 광각 이미지
 - 모델은 train data에서는 잘 예측하지만 test data에서는 잘 예측하지 못 함

Domain adaptation

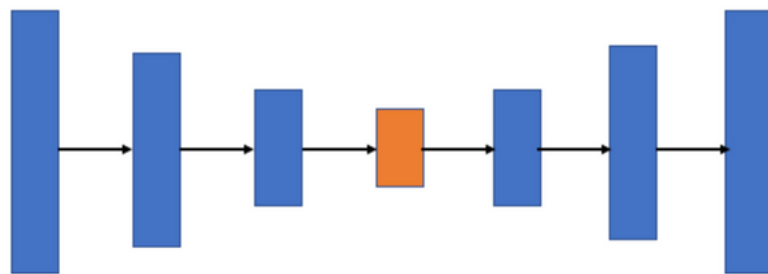
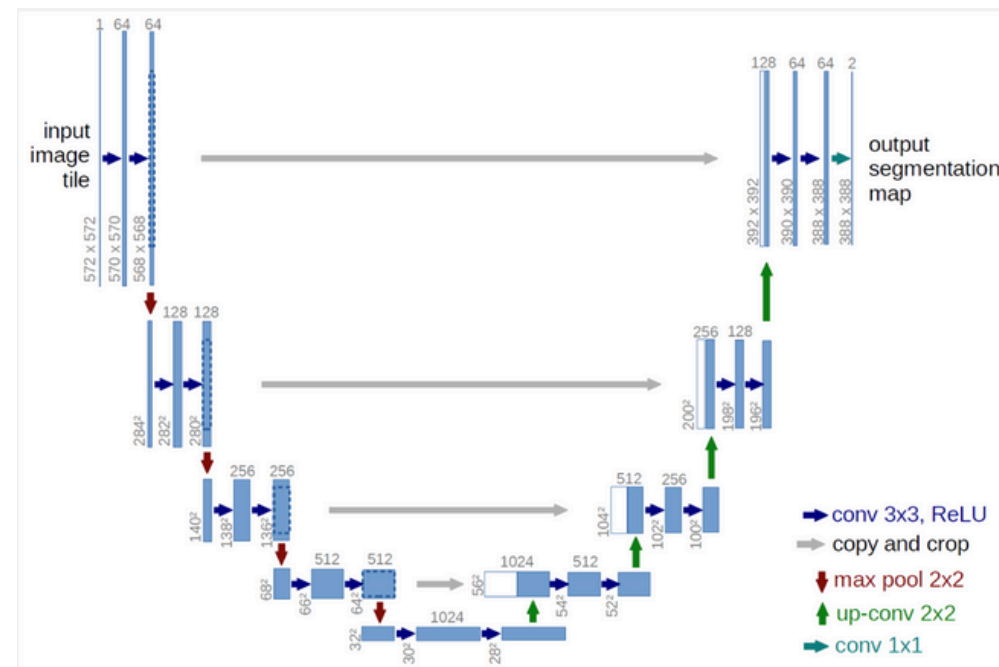
- 레이블이 불충분하거나 존재하지 않는 target domain (혹은 label에 대한 cost가 매우 높은 경우)에 대해서 레이블이 풍부하고 target domain과 관련이 있는 source domain을 이용해서 target domain에도 효과적으로 추론할 수 있도록 함

Self-training



- Pseudo label(가짜 라벨)을 통해서 스스로 학습하는 알고리즘
- 모델의 이전 상태에서 출력된 예측값을 target domain에 대한 pseudo label로 보고 이를 GT처럼 여기며 현재 모델을 학습시키는 것
- Self-training을 위해서는 두 개의 모델이 필요
 - pseudo label을 생성하기 위한 모델 (**teacher model**)
 - pseudo label을 학습하고 가중치를 업데이트하는 **student model**
- 학습 과정
 - 레이블이 달린 데이터로 모델을 학습
 - 이 모델을 가지고 레이블이 달리지 않은 데이터를 예측
 - 이 중에서 가장 확률값이 높은 데이터들만 레이블 데이터로 다시 가져간다.
 - 위 과정을 계속 반복

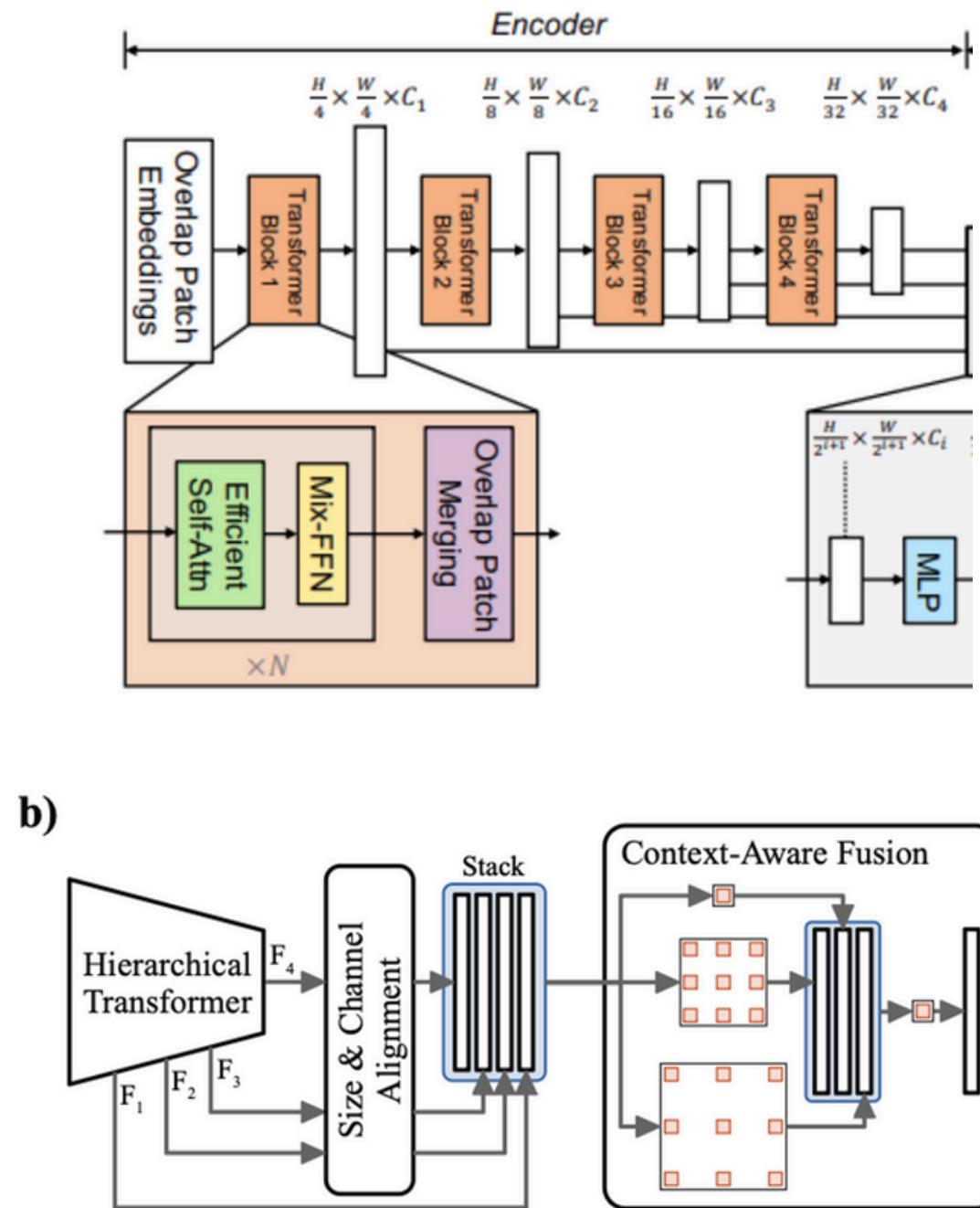
3. 모델 소개



U-Net

- Baseline Model
- 인코더-디코더 기반의 모델
 - 인코딩 단계
 - 입력 이미지의 특징을 포착할 수 있도록 채널의 수를 늘리면서 차원을 축소해 나감.
 - 디코딩 단계
 - 저차원으로 인코딩 된 정보만을 이용하여 채널의 수를 줄이고 차원을 늘려서 고차원의 이미지를 복원
- 인코딩 단계의 각 레이어에서 얻은 특징을 디코딩 단계의 각 레이어에 합치는 방법 사용

3. 모델 소개



DAFormer

- Domain adaptation에 적합한 모델

- 인코더

- semantic segmentation에 적합한 MixTransformer 사용
- 이미지를 4x4 크기의 작은 패치로 나눠 self attention 수행
- Multi-level feature map을 생성하고 로컬 연속성을 유지하기 위해 다운 샘플링은 overlapping patch merging 방식 사용

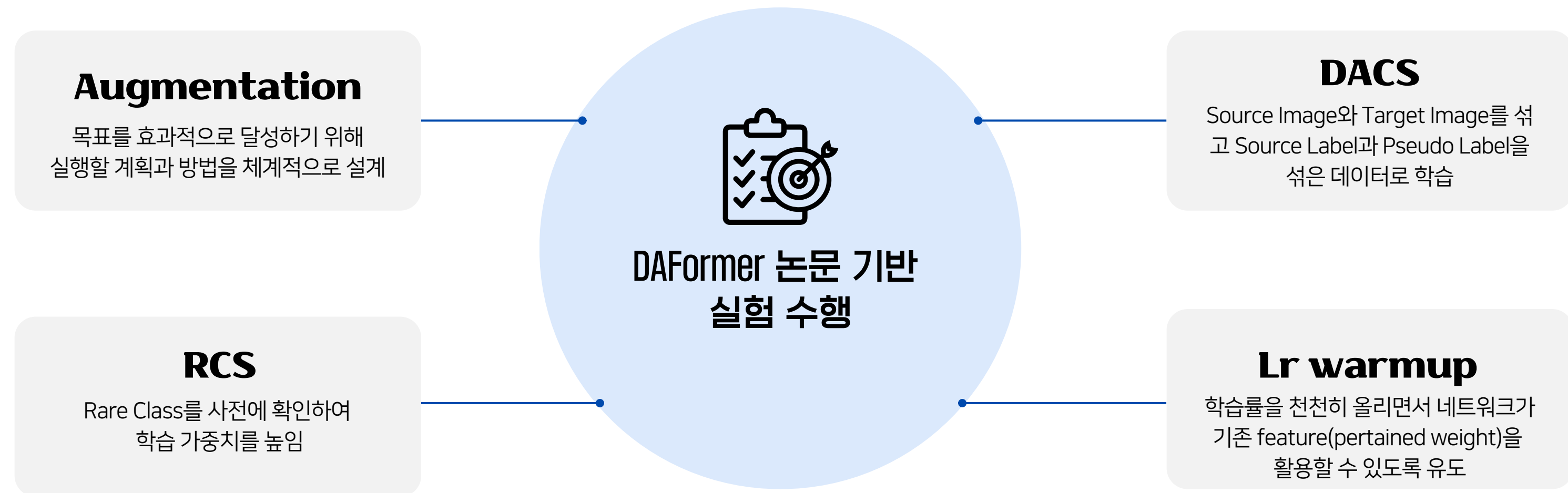
- 디코더

- 인코더에서 만들어진 4개의 feature map을 모두 사용
- 낮은 해상도의 feature는 가장 높은 해상도에 맞추어 bilinear upsampling 후 모든 feature를 concatenate

- RCS : 초기 학습에서 빈도 높은 클래스 위주로 학습 편향이 생기므로, 드문 클래스를 더 자주 뽑히도록 함
- Thing class ImageNet : source data에 과적합 되지 않도록 source의 feature가 image net feature와 비슷하게 나오도록 유도
- Learning rate warm up : 학습률을 천천히 올리면서 네트워크가 기존 feature(pertained weight)을 활용할 수 있도록 유도

4. 적용한 기법

성능 향상을 위해 적용한 기법



4. 적용한 기법

성능 향상을 위해 적용한 기법

Augmentation

original image



barrel distortion

- Target 이미지 특성을 반영해 왜곡 환경에 적응력 향상

gaussian blur

- 흐릿한 이미지 환경에서도 인식 성능을 유지하도록 학습



color jitter

- 밝기·대비·채도 변화를 주어 다양한 조명 조건에 강건하게 학습

Learning rate warm up

: 학습 초기에 학습률을 점진적으로 증가시켜 모델이 안정적으로 수렴하도록 지원

4. 적용한 기법

성능 향상을 위해 적용한 기법

RCS

결과:

Class 0 - Pixels: 746,045,273 (16.0678%)
Class 1 - Pixels: 57,343,899 (1.2350%)
Class 2 - Pixels: 910,012,067 (19.5993%)
Class 3 - Pixels: 70,083,493 (1.5094%)
Class 4 - Pixels: 27,331,168 (0.5886%)
Class 5 - Pixels: 9,218,291 (0.1985%)
Class 6 - Pixels: 29,086,677 (0.6265%)
Class 7 - Pixels: 667,439,162 (14.3749%)
Class 8 - Pixels: 1,059,062,740 (22.8094%)
Class 9 - Pixels: 5,085,820 (0.1095%)
Class 10 - Pixels: 2,071,203 (0.0446%)
Class 11 - Pixels: 452,013,131 (9.7352%)
Class 12 - Pixels: 608,301,604 (13.1012%)

DACS

- 전체 픽셀 비중이 1%미만: 4, 5, 6, 9, 10

```
# 클래스 픽셀 수 카운팅 디렉터리
class_pixel_counts = defaultdict(int)

# 13개 클래스 (0~12)
num_classes = 13

# 이미지별 rare class 포함 여부 저장
sample_has_rare_class = []

# rare class 기준
rare_classes = [4, 5, 6, 9, 10] # 비율 < 1%인 클래스들

for idx in range(len(df)):
    gt_path = df.iloc[idx]['gt_path']
    mask = cv2.imread(gt_path, cv2.IMREAD_GRAYSCALE)

    unique, counts = np.unique(mask, return_counts=True)
    class_counts = dict(zip(unique, counts))

    for c, count in class_counts.items():
        class_pixel_counts[c] += count

    has_rare = any([c in rare_classes for c in class_counts])
    sample_has_rare_class.append(has_rare)

df['has_rare'] = sample_has_rare_class

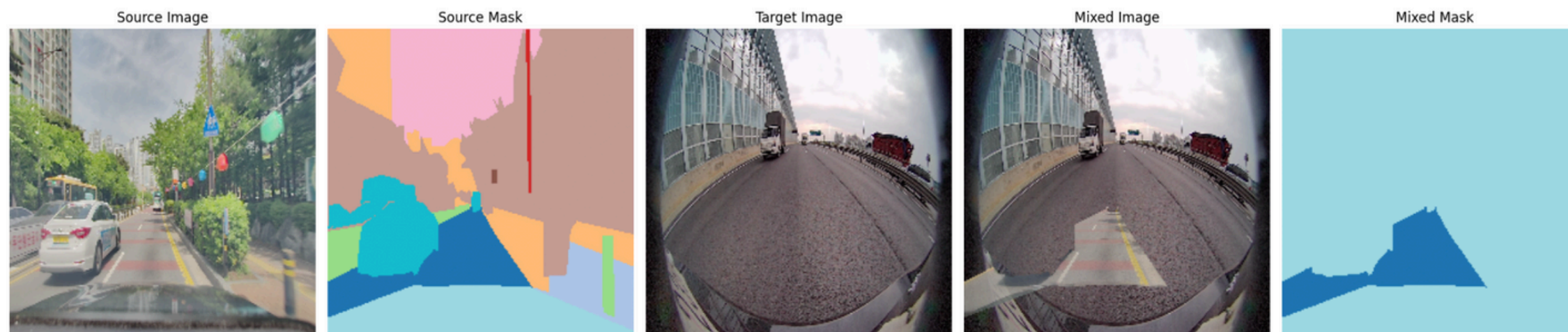
# rare class 포함된 샘플은 가중치 높게
weights = [3.0 if has_rare else 1.0 for has_rare in df['has_rare']]

# numpy -> tensor
sample_weights = torch.DoubleTensor(weights)

sampler = WeightedRandomSampler(weights=sample_weights,
                                num_samples=len(sample_weights), replacement=True)
```

: 기존의 Naive Mixing(target domain에서 두 이미지를 뽑아 mix)과는 다르게 source domain과 target domain을 섞는 방식

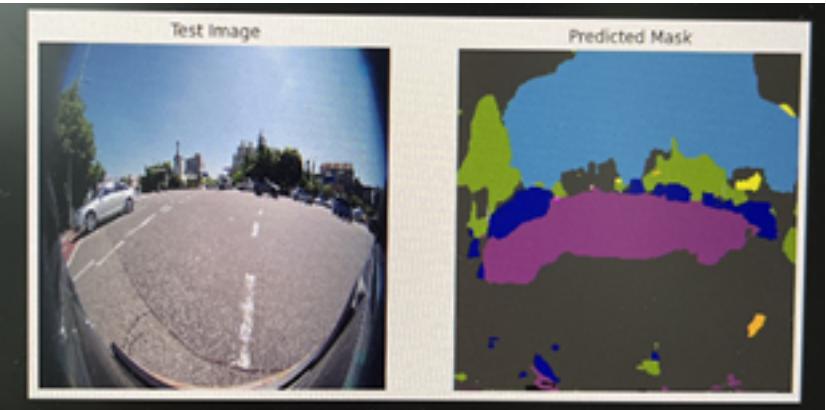
- 구현은 하였지만 실제로 적용 X



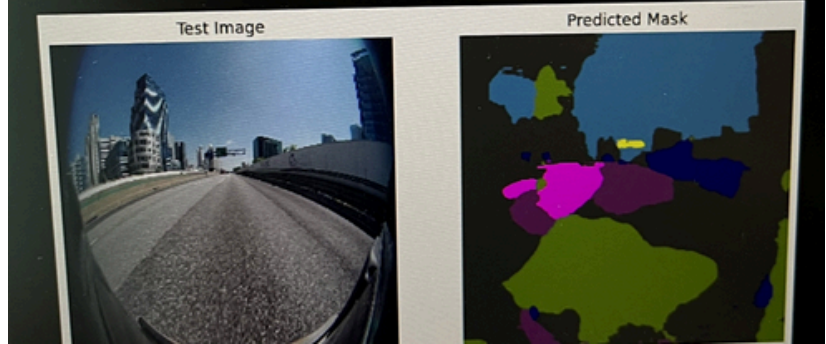
5. 실험 과정

U-Net 실험과정

	성능	Input	RCS	Loss	Epoch
1	0.16	Source+barrel	X	Cross entropy	20
2	0.17	Source	X	Cross entropy	20
3	0.2	Source+barrel	O	Crossentropy Road weight=5	200
4	0.24	Source+barrel	O	Dice loss Road weight=5	300



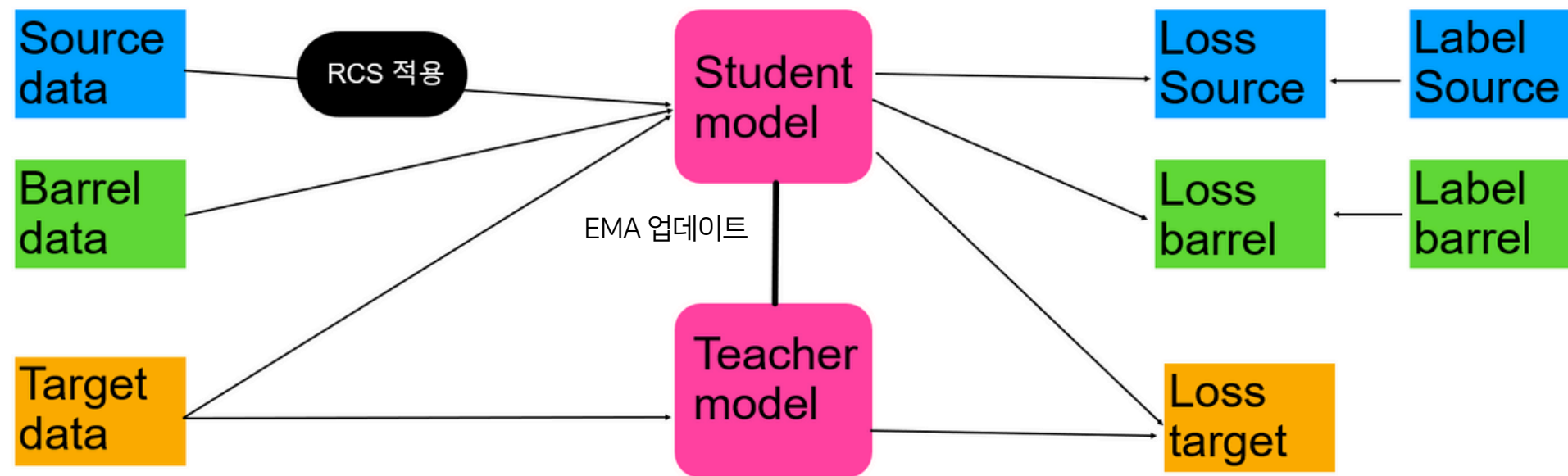
• 3번 시각화



• 4번 시각화

5. 실험 과정

DAFormer 실험과정



- Pretrained weight - cityscape
- Epoch 100
- (224, 224)
- 결과: 0.06

5. 실험 과정

DAFormer

(실험 1) augmentation 된 이미지가 오히려 domain shift를 더 심하게 일으켰다고 가정
→ source만 넣고 train 시켰지만 큰 변화 X

(실험 2) epoch 수 올리기
→ epoch 수를 올렸을 때 소폭 상승했지만 큰 변화 X

(실험 3) input 이미지의 해상도와 인코더의 성능을 b1으로 높임



→ 성능이 약 두 배 정도 향상
그러나 가장 중요한 road를 pole로 인식함
• 결과: 0.17

5. 실험 과정

DAFormer

(실험 3-1) 인코더의 성능을 b5까지 확인하려 함
→ 숫자가 높아질수록 OOM 문제 발생
(b1도 다시 할려니 OOM 문제가 발생해서 그 이후에는 b0로 사용)

(실험 3-2) RCS를 제외함
RCS로 인해서 road를 rare class인 pole로 학습편향이 일어났다는 가정
→ 그러나, 여전히 road를 인식하지 못함

(실험 3-2-1) loss 함수에서 road의 가중치를 높임

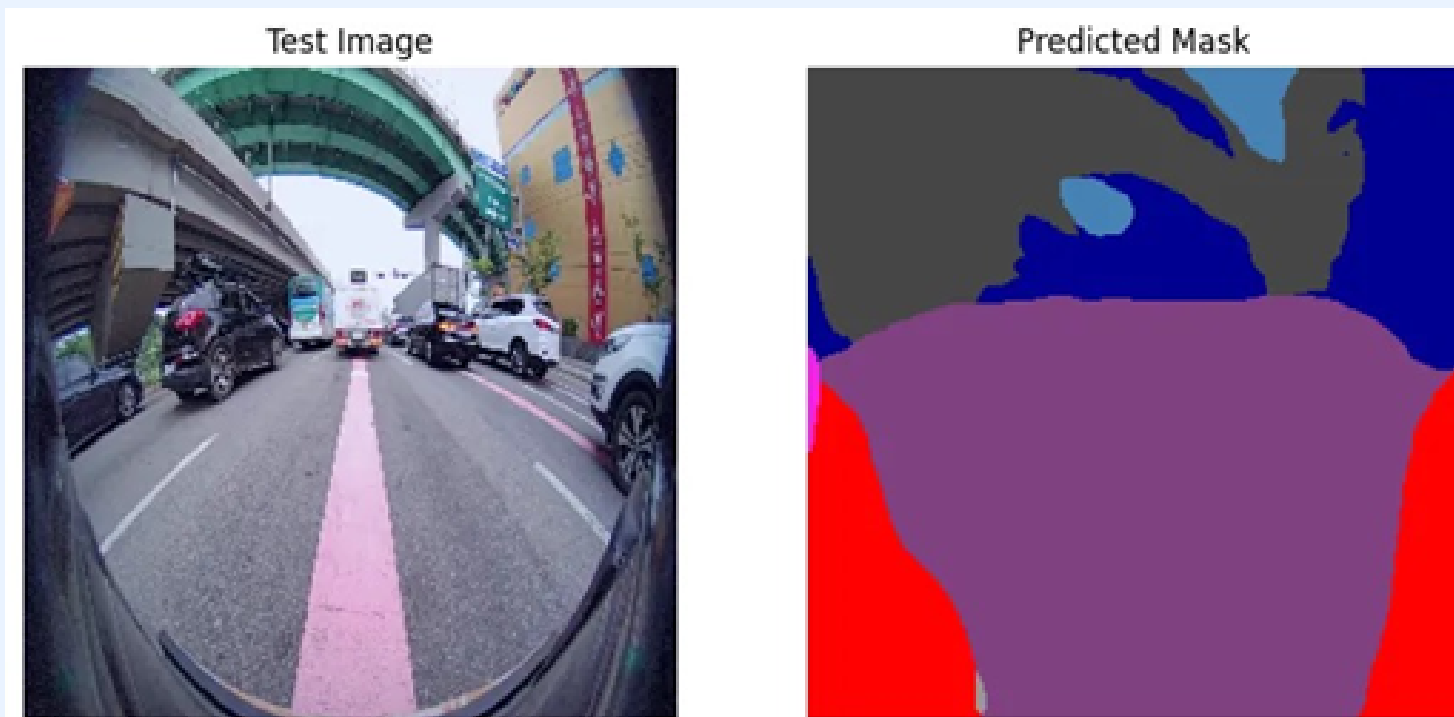


→ 유의미한 상승
• 결과: 0.13

5. 실험 과정

DAFormer

(실험 4) 디코더 변경 → 유의미한 상승



- 결과: 0.12

(실험 5) learning rate warmup 적용 → 성능 하락

(실험 4-1) target 이미지의 background를 잘 인식하지 못함



- barrel distortion augmentation한 이미지를 target 이미지와 비슷하게 수정 & loss함수에서 background 가중치 높임
- background는 잘 인식하지만 그 외는 잘 예측하지 못함
- 결과: 0.13

6. 결론

결론

daformer의 성능이 예상과 달리 낮게 나온 이유

- pseudo-label의 퀄리티 문제
- 잘못된 pseudo-label은 모델을 오히려 악화시킬 수 있음
초기모델이 나쁘면, 잘못된 정보로 점점 학습이 망가지고 오류가 점점 커짐



추가 실험 제안

- Confidencethresholding 적용하여 pseudo-label 중 신뢰도 높은 샘플만 사용
- DiceCELoss 적용
- DACS 적용
- MiT-b5, (1024,1024)로 적용

감사합니다

유런 8기 프로젝트
참치팀